SIAM: Institutional Admission and Enrollment System

***Farit Reasco [1], Ricardo Plasencia [2]**

[*1] Pontificia Universidad Catolica Esmeraldas; fareasco@pucese.edu.ec
[2] Overseer Investigation Center; Ricardo.plasencia@overseeric.com

Abstract: This paper presents the design and implementation of the *Institutional Admission and Enrollment System (SIAM)* developed for the Don Bosco Fiscomisional Educational Unit. The system centralizes and automates the student admission process, including application management, quota administration, leveling courses, payment validation, and final enrollment. The solution is built using a monorepo architecture with NestJS as the backend framework, Next.js 14 for the frontend, PostgreSQL as the relational database engine, and Docker for containerization. A hierarchical role-based access control (RBAC) model with five privilege levels was implemented, along with authentication using JSON Web Tokens (JWT). The results demonstrate a functional, scalable, and secure platform that reduces administrative processing times and improves the traceability of the admission workflow.

Keywords: educational; information systems; school admission; role-based access control REST API; NestJS; Next.js; PostgreSQL; Docker; JWT; academic management.

1. Introduction

The admission and enrollment process in primary and secondary education institutions is often supported by physical forms, isolated spreadsheets, and fragmented records. This situation increases processing times, the likelihood of data entry errors, and the difficulty of administratively tracking students [2], [7]. In several Latin American contexts, the absence of integrated academic management systems has been shown to generate inefficiencies in student registration, course allocation, and the control of critical information. This negatively affects both the experience of families and the quality of data available for institutional decision-making [4], [11].

In response to these challenges, various countries have implemented centralized school admission platforms that regulate quotas transparently, automate application registration, and structure enrollment workflows through standardized processes supported by robust information systems [1], [3].

Additionally, institutional guidelines recommend that critical processes such as admission and enrollment be supported by digital tools that enable the consolidation of historical data, monitoring of management indicators, and reduction of dependency on in-person procedures [5], [6].

In the specific case of the Don Bosco Fiscomisional Educational Unit, the sustained increase in demand for vacancies in Basic General Education and High School levels, together with the need to manage leveling courses, payment validation, and class assignments, highlights the necessity of a unified platform that integrates the entire admission and enrollment lifecycle. The absence of an institutional system incorporating role-based access control, automated document validation, and real-time reporting limits both academic and administrative supervision while increasing the operational workload of administrative staff [2], [7].

The *Institutional Admission and Enrollment System (SIAM)* is proposed as a specialized web platform that automates the application workflow from the guardian's initial registration to the

student's final enrollment. The system integrates functionalities such as quota management, course administration, payment validation, and official document generation.

The solution is based on a modern web architecture using a REST API, JWT-based authentication, and role-based access control (RBAC), following best practices in software architecture for educational applications and academic management systems [8], [13]–[15].

1. Introduction

The admission and enrollment process in primary and secondary education institutions is often supported by physical forms, isolated spreadsheets, and fragmented records. This situation increases processing times, the likelihood of data entry errors, and the difficulty of administratively tracking students [2], [7]. In several Latin American contexts, the absence of integrated academic management systems has been shown to generate inefficiencies in student registration, course allocation, and the control of critical information. This negatively affects both the experience of families and the quality of data available for institutional decision-making [4], [11].

2. Objectives

2.1. General objective

To design and implement the *Institutional Admission and Enrollment System (SIAM)* for the Don Bosco Fiscomisional Educational Unit, integrating into a unified web platform the management of admission applications, quota administration, admission courses, payment validation, and final enrollment, ensuring traceability, security, and availability of academic and administrative information.

2.2. Specific objectives

a) To analyze the current admission and enrollment process, identifying actors, information flows, bottlenecks, and recurring issues that justify automation through an information system.
b) To design the software architecture of SIAM under a modular web application approach, clearly defining components, modules, and data flows between the backend (NestJS/PostgreSQL) and the frontend (Next.js).
c) To implement a secure REST API for managing the lifecycle of admission applications, quotas, courses, and enrollments, integrating JWT-based authentication and RBAC aligned with the institution's organizational structure.
d) To deploy the system in a containerized environment using Docker and Docker Compose, documenting requirements, installation steps, environment variables, and database initialization procedures [8], [14].
e) To develop a responsive web interface for guardians and administrative staff that allows application management, status monitoring, course administration, and real-time report generation.
f) To document the structure of the monorepo and its main modules to facilitate system maintenance and scalability [13], [14].

3. Scope

The scope defines the functionalities implemented and evaluated during this final degree project, as well as the elements considered outside the target version. This delimitation is based on technical feasibility, available time, and institutional priorities.

3.1. Included and Excluded Elements

The system includes:

- Full application lifecycle (DRAFT → MATRICULATED)
- User management and hierarchical RBAC [9], [12].
- Quota administration by level, schedule, and class [1], [3].
- Course management (scheduling, attendance, grading)
- Payment validation and receipt processing
- Administrative reports and PDF document generation [4], [8].

The system includes:

- Full application lifecycle (DRAFT → MATRICULATED)
- User management and hierarchical RBAC
- Quota administration by level, schedule, and class
- Course management (scheduling, attendance, grading)
- Payment validation and receipt processing
- Administrative reports and PDF document generation

3.2. General objective

The development of SIAM is constrained using open-source technologies to avoid licensing costs and facilitate deployment in resource-limited institutional environments.

It is assumed that the institution has stable internet connectivity for administrative staff and that guardians have access to a web-enabled device [2], [7], [8]. Additionally, initial system configuration (academic periods, quotas, levels) will be performed by administrative users.

4. System Requirements

4.1. Software Requirements

The system follows a client-server architecture where the backend exposes REST services and the frontend consumes them, consistent with modern web application practices [14], [18].

Backend technologies include Node.js, NestJS, TypeScript, PostgreSQL, Prisma ORM, JWT authentication, and Swagger documentation [2], [4], [17].

Frontend technologies include Next.js, React, Tailwind CSS, and modern UI libraries for responsive design.

Infrastructure is based on Docker and Docker Compose, enabling reproducible environments [8], [14].

4.2. Hardware Requirements

A basic deployment requires a server with sufficient processing power, memory, and storage, following recommendations for academic systems [4], [8], [16].

4.3. External Dependencies

The system depends on email services, persistent storage, and optional external validation services for future integrations.

4.4. Non-Functional Requirements

- Performance under concurrent loads

- Security through JWT and RBAC [9], [12], [17]
- High availability during admission periods
- Maintainability through modular design [13], [14], [16]
- Usability with intuitive interfaces [2], [8], [11]

5. System Architecture

The architecture of the SIAM system is conceived as a three-layer web application (presentation, business logic, and data), deployed under a monorepo structure that integrates the backend, frontend, and technical documentation within a single repository. This approach facilitates consistency between layers, reuse of shared types, and unified management of dependencies and deployment scripts, in accordance with modern software architecture practices for web-based educational systems [13]–[15].

5.1. Overview of the Solution

At a high level, the system is composed of the following elements:

- **Web Client (Next.js):** Responsible for presentation and interaction with end users. It consumes the backend REST API and manages sessions using NextAuth and JWT.
- **REST API (NestJS):** Implements the business logic of the admission and enrollment process, orchestrates the application lifecycle, and exposes endpoints protected by authentication and RBAC.
- **Database (PostgreSQL + Prisma):** Stores structured data including users, applications, quotas, courses, configurations, and audit logs.
- **Infrastructure Services:** Includes Docker containers, persistent volumes, and SMTP servers for transactional email communication.

Communication between frontend and backend is performed through HTTPS using a stateless REST API, following principles such as client-server separation, resource identification, and standardized operations, which contribute to system scalability and simplicity [14], [18].

5.2. Backend Modules

The backend is structured into modules aligned with the system's functional domains, following separation of concerns and layered architecture principles [13], [15], [16]:

- **auth:** Handles authentication through credentials, JWT generation and validation, guards, and role-based decorators [17].
- **users:** Manages the user lifecycle, including creation, administrative approval, updates, deactivation, and role assignment.
- **roles / RBAC:** Implements a hierarchical model with five roles and inherited permissions, aligned with RBAC models applied in educational environments [9], [12].
- **applications:** Core module of the system; manages the lifecycle of admission applications (from draft to enrollment), document attachments, correction workflows, and continuity logic for returning students.
- **quotas:** Controls seat availability by level, schedule, and class group, with real-time occupancy monitoring [1], [3], [4].
- **cursillo:** Manages preparatory courses, including scheduling, attendance tracking, and applicant evaluation.
- **system-config:** Handles global system parameters such as academic year, admission periods, and fee structures, allowing system behavior adjustments without modifying source code.
- **notifications:** Sends automated emails triggered by relevant events such as account creation, status changes, and correction requests.
- **audit:** Records critical user actions to ensure traceability and compliance with data governance practices [6], [8].

5.3. Frontend Architectur

The frontend is structured into functional segments based on user roles, ensuring modularity and usability [14], [18]:

- **Public Section:** Provides general information and access to login and registration.
- **Guardian Portal:** Allows creation and management of applications, document uploads, status tracking, and payment submission.
- **Administrative Panel:** Provides differentiated views for roles such as superadmin, admin, principal, and secretary, including dashboards, reports, and configuration tools.
- **Shared Components:** Built using Shadcn/UI, including multi-step forms, exportable tables, and modal interfaces, ensuring visual consistency and component reuse [8], [11].

5.4. Technical Justification of the Architecture

The adoption of a REST API-based architecture with a decoupled frontend and modular backend responds to key requirements such as scalability, maintainability, and adaptability, widely supported in software architecture literature for educational systems [4], [13], [14], [16].

This design allows the incorporation of new functionalities without restructuring the system, facilitating future evolution toward a microservices architecture if system complexity or usage demand increases [14].

The implemented admission workflow follows a state machine model defined as:

DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED → (course if applicable) → PAYMENT_UPLOADED → PAYMENT_VALIDATED → MATRICULATED

This model ensures process traceability, consistency in state transitions, and alignment with digital admission system practices [1]–[3], [7].

6. Installation and Configuration

The installation and configuration of the SIAM system are documented to ensure that a third party can deploy the system in a clean environment without additional assistance, following best practices for reproducibility in academic software solutions [8], [14].

6.1. Prerequisites

The following requirements must be met before deploying the system:

GNU/Linux operating system (recommended), macOS, or Windows with Docker Desktop installed

Docker and Docker Compose properly installed and configured

Git for repository cloning

Internet connection for downloading base images and dependencies

6.2. Environment Variable Configuration

Proper configuration of environment variables is required for both backend and frontend components.

6.2.1. Backend (backend/.env)

DATABASE_URL: Connection string to the PostgreSQL database

JWT_SECRET: Secret key for signing authentication tokens (must be secure and randomly generated) [17]

PORT: Server port (default: 4000)

MAIL_HOST, MAIL_PORT, MAIL_USER, MAIL_PASS: SMTP configuration parameters

6.2.2. Frontend (frontend/.env.local)

NEXT_PUBLIC_API_URL: Base URL of the backend API

NEXTAUTH_URL: Public URL of the frontend application

NEXTAUTH_SECRET: Secret key for encrypting session data

6.3. Deployment with Docker Compose

To deploy the system using Docker Compose, execute the following steps from the root of the monorepo:

1. Build and start containers in detached mode:
docker-compose up --build -d
2. Apply database migrations:
docker exec -it siam-backend npx prisma migrate dev
3. (Optional) Load seed data:
docker exec -it siam-backend npx prisma db seed

This approach ensures reproducibility, portability, and consistency across development and production environments [8], [14].

6.4. Local Installation without Docker

6.4.1. Backend Setup

Install PostgreSQL and create the target database
cd backend && npm install
Configure the .env file with the local database connection
npx prisma generate
npx prisma migrate dev
npm run start:dev

6.4.2. Frontend Setup

cd frontend && npm install
Configure .env.local with the backend URL
npm run dev

6.5. Post-Installation Verification

Frontend: http://localhost:3000
REST API: http://localhost:4000
Swagger Documentation: http://localhost:4000/api/docs
These verification steps ensure correct system setup and functionality [5], [8].

7. System usage

7.1. Authentication and Session Management

Guardians register through the public portal and, after administrative approval, gain access to their dashboard. Staff access the administrative panel with institutional credentials. After authentication, the backend issues a JWT token used in each request (Authorization: Bearer <token>) [17]. RBAC determines permissions per user [9], [12].

7. System usage

7.2. Admission Workflow — Guardian Portal

Draft: Multi-step form with student and family data (DRAFT).
Document Upload: Upload required files.
Submission: Application moves to SUBMITTED.
Corrections: Admin may request changes.
Tracking: Real-time status monitoring.
Payment: Upload receipt after approval.

7.3. Administrative Workflow — Management Panel

Secretary: Reviews applications, validates payments, assigns classes.
Principal: Approves/rejects applications, views reports [6], [11].
Administrator: Configures system and users.
Superadmin: Controls global system settings [9], [12].

7.4. Reports and Dashboards

Application statistics by status and level
Real-time quota tracking
Export to CSV/XLSX
PDF document generation

8. Repository structure

The system follows a monorepo architecture integrating backend, frontend, and documentation [13], [14].

8.1. Directory Structure

```
siam/
├── backend/
├── frontend/
├── docs/
├── docker-compose.yml
├── README.md
└── .gitignore
```

8.2. Main Components

Backend: NestJS API, Prisma, modules.
Frontend: Next.js app with UI components.
Docs: Technical documentation.
Root: Docker, README, configuration files.

9. Conclusions and recommendations

The development of the *Institutional Admission and Enrollment System (SIAM)* for the Don Bosco Fiscomisional Educational Unit demonstrates that the comprehensive automation of the admission and enrollment process—from initial application registration to final student enrollment—significantly contributes to reducing administrative processing times, minimizing data entry errors, improving process traceability, and enhancing institutional decision-making through real-time indicators [2], [4], [11].

The implemented architecture, based on NestJS for the backend, Next.js for the frontend, PostgreSQL with Prisma as the persistence layer, and Docker for containerization, provides an effective balance between system complexity, maintainability, and scalability. This technological combination aligns with well-established software architecture practices for modern web-based educational systems, ensuring long-term sustainability and adaptability [13]–[16].

Furthermore, the incorporation of a hierarchical role-based access control (RBAC) model, together with JWT-based authentication mechanisms, guarantees the security and confidentiality of sensitive information. By applying the principle of least privilege and integrating auditing capabilities, the system enables precise control over user actions and strengthens institutional data governance practices [9], [12], [17].

The use of Docker and Docker Compose as deployment strategies proved to be a robust solution for achieving reproducible, portable, and easily maintainable environments. This approach reduces

deployment complexity and ensures consistency between development and production environments, which is critical in institutional systems with evolving requirements [8], [14].

In addition, the modular design of the system and its monorepo structure facilitate future extensions, allowing the incorporation of new features without requiring major architectural changes. This characteristic is essential for systems intended to evolve alongside institutional needs and technological advancements [13], [14].

9.1. Future Work

Based on the results obtained, several lines of future development are proposed:

- **Integration with national education systems:**
  Establish interoperability with Ecuador's Ministry of Education platforms to automate validation processes, reduce data duplication, and enhance institutional data consistency, following successful implementations of centralized admission systems in the region [1], [3].
- **Expansion toward full academic management:**
  Extend the system to include modules for grading, attendance tracking, discipline management, and advanced analytics, enabling a comprehensive academic management platform [4], [6], [11].
- **Implementation of automated testing strategies:**
  Incorporate unit, integration, and performance testing frameworks with coverage metrics and traceability to ensure system reliability under different operational conditions [13], [16].
- **Multi-institution scalability:**
  Adapt the system's configuration modules and RBAC model to support multi-tenant deployments across multiple educational institutions, leveraging its current scalable architecture [14].
- **Advanced data analytics and reporting:**
  Integrate business intelligence tools and predictive analytics models to support strategic decision-making and improve educational quality indicators.

*References

1. Ministerio de Educación de Chile. Sistema de Admisión Escolar: Claves para apoyar el proceso de admisión escolar desde la gestión de los Servicios Locales; Santiago de Chile, 2025. Available online: https://educacionpublica.gob.cl/wp-content/uploads/2025/04/Orientaciones-Claves-SAE-y-NEP.pdf
2. Herrera, M. et al. Diseño de una aplicación web ASP.NET para la gestión de matrícula y registro de calificaciones. Revista Investigación y Desarrollo 2024. Available online: http://ve.scielo.org/scielo.php?script=sci_arttext&pid=S2665-03982024000202021
3. Ministerio de Educación de Chile. Protocolo de manejo de datos del Sistema de Admisión Escolar; Santiago de Chile, s.f. Available online: https://cdnsae.mineduc.cl/docs/protocolo_de_datos.pdf
4. Ortegón Cortázar, G. Optimización de sistemas de gestión académica: una propuesta de gestión, medición y procesamiento de datos en un entorno virtual de aprendizaje. Revista EAN 2015, 79, 45–63. Available online: http://www.scielo.org.co/scielo.php?script=sci_arttext&pid=S0120-81602015000200006
5. Gobierno de la Provincia de Santa Fe. Sistema de Gestión Educativa: Manual de Usuario — Módulo Alumnos; Santa Fe, Argentina, s.f. Available online: https://www.santafe.gov.ar/index.php/educacion/content/download/167197/813272/file/06.%20M+%C2%A6dulo%20Alumnos-V1.pdf
6. García, A.; Ballester, L. La reingeniería administrativa en una institución de educación superior: uso de macrodatos para la gestión académica. Revista de Investigación Educativa 2020, 20. Available online: https://www.scielo.org.mx/article_plus.php?pid=S1665-26732020000100045&tlng=es&lng=es
7. Universidad Politécnica Salesiana. Desarrollo de la aplicación web para el registro de matrículas y gestión de estudiantes en la Escuela "José Martí"; Tesis de grado, Quito, Ecuador, s.f. Available online: https://dspace.ups.edu.ec/bitstream/123456789/20951/1/UPS-GT003386.pdf

8. Lamz Piedra, R. et al. Sistema web para la gestión de la superación profesional en instituciones educativas. Revista Cubana de Educación Superior 2021, 40(1). Available online: http://scielo.sld.cu/scielo.php?script=sci_arttext&pid=S1684-18592021000100015
9. Mora, J.; Alonso, P. Modelo de control de acceso X-RBAC para un framework de aprendizaje colaborativo basado en web. Revista Iberoamericana de Tecnologías del Aprendizaje 2014. Available online: https://www.academia.edu/486983/MODELO_DE_CONTROL_DE_ACCESO_X_RBAC_PARA_UN_FRAMEWORK_DE_APRENDIZAJE_COLABORATIVO
10. Universidad Piloto de Colombia. Sistema de información para el proceso de matrículas, control de asistencias y gestión de notas de los estudiantes; Trabajo de grado, Bogotá, Colombia, s.f. Available online: https://repository.unipiloto.edu.co/bitstream/handle/20.500.12277/10465/Proyecto__grado_final_Manuel_Diaz.pdf
11. Rosillo Querevalú, R. Gestión académica, automatización y sistemas de información en instituciones educativas. Revista Pakamuros 2023, 5(2). Available online: https://revistas.unj.edu.pe/index.php/pakamuros/article/view/484/600
12. Universidad Politécnica de Valencia. Análisis y diseño de un sistema de control de acceso basado en roles (RBAC) para entornos web; Trabajo de fin de grado, Valencia, España, s.f. Available online: https://riunet.upv.es/bitstreams/178166b7-044c-4c47-ba19-5981edf49d89/download
13. Piattini, M. Diseño de la arquitectura de software. En Ingeniería de Software: Un enfoque práctico; Editorial universitaria: Madrid, España, 1996. Available online: https://www.academia.edu/90329826/DISE%C3%91O_DE_LA_ARQUITECTURA_DE_SOFTWARE
14. Rosado Castellanos, D.U.; Pacheco Farfán, I.S.; Fuentes Chab, I.H.; Cantun Páez, J.C. Arquitectura de software para el desarrollo de aplicaciones web orientada a micro-servicios en TecNM campus Escárcega. ProgMat 2023, 15(2), 11–24. Available online: https://progmat.uaem.mx/progmat/index.php/progmat/article/view/v15n02a02
15. García, A.; Ballester, L. Arquitectura de software para el producto "Mis Mejores Cuentos". En Memorias del Congreso Internacional de Educación de Calidad (EDUQA); 2012.
16. Hernández Bernal, H. Análisis de desempeño de una arquitectura de software para aplicaciones web; Tesis de licenciatura, Universidad Nacional Autónoma de México, México D.F., 2010.
17. Parra, O. Análisis de sistemas de autenticación y autorización para entornos distribuidos mediante OAuth 2.0 y JWT; Trabajo final de máster, Universitat Oberta de Catalunya, Barcelona, España, 2020. Available online: https://openaccess.uoc.edu/bitstream/10609/107926/6/oparrabTFM0120memoria.pdf
18. Universidad Técnica del Norte. Análisis de frameworks de desarrollo de API REST y su aplicación en sistemas web empresariales; Proyecto de grado, Ibarra, Ecuador, s.f. Available online: https://repositorio.utn.edu.ec/bitstream/123456789/8264/1/PG%20659%20TESIS.pdf